SP1 zkVM Architecture

Succinct Labs

April 28, 2026

Overview

This document describes the architecture of the **SP1 zkVM** according to the W1 deliverable of the EF zkVM whitepaper guidelines[1]. SP1 proves execution of RISC-V guest programs using:

- Execution of RV64IM programs compiled from ELF binaries
- Sharded execution traces with chip-cluster–based segmentation
- Specialized constraint systems ("chips") grouped into clusters
- Local bus interactions (LogUp + GKR) within each shard
- Global bus interactions across shards (septic curve accumulation)
- Hypercube proof system with BaseFold polynomial commitments over the KoalaBear field
- Recursive aggregation via normalize / compose / shrink circuits

After execution, the trace is partitioned into *shards*, each corresponding to a chip cluster instance. Every shard is proven independently, producing a core shard proof. These proofs are then normalized to a fixed shape and recursively aggregated until a single proof attests to the correctness of the entire program execution.

1 Execution Model

We define how a program is executed.

1.1 Program, Execution, and VM State

A **program** $\mathcal{P}$ is a RISC-V ELF binary targeting the RV64IM instruction set (64-bit base integer with multiply/divide extension). Its inputs $\mathcal{I}$ are derived from the Ethereum state (*initial state*) and the execution witness $\mathcal{W}$, and its outputs $\mathcal{O}$ are the final Ethereum state (*final state*).

The program $\mathcal{P}$ operates on the **VM state** S, defined as:

- `pc`: 64-bit program counter
- `regs`[0..31]: 32 general-purpose registers (x0–x31), each 64 bits wide, with x0 hardwired to zero

[1] `https://zkevm.ethereum.foundation/blog/cryptography-research-update`

- $\texttt{mem}[]$: byte-addressed writable memory

We separately denote the code of the program by `code`. It is immutable and read-only, and is not part of the VM state.

A single instruction step of the program execution is defined as a transition from one VM state to another according to the semantics of the instruction op retrieved from `code` at the program counter $\texttt{pc}_1$. This is denoted as follows:

$$S := (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1) \xrightarrow{\text{op}:=\texttt{code}[\texttt{pc}_1]} S' := (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2) \tag{1}$$

For example, if op is an addition instruction, which adds the first two registers and stores in the third register, then the semantics of the step is defined as

$$\texttt{pc}_2 = \texttt{pc}_1 + 1, \tag{2}$$

$$\texttt{regs}_2 = \texttt{regs}_1 \text{ except that } \texttt{regs}_2[2] := \texttt{regs}_1[0] + \texttt{regs}_1[1], \tag{3}$$

$$\texttt{mem}_2 = \texttt{mem}_1. \tag{4}$$

We will use the notation $(\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1) \to (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$ to denote that S transitions to S' by executing instruction $\text{op} := \texttt{code}[\texttt{pc}_1]$.

An **execution trace** for program `code` applied to inputs $\mathcal{I}$ is a sequence of VM states

$$\mathcal{T}: \quad S_0 \to S_1 \to \cdots \to S_T$$

where each step corresponds to execution of one instruction.

A **zkVM proof** for $(\mathcal{P}, \mathbb{E}, \mathbb{E}')$, where $\mathbb{E}$ and $\mathbb{E}'$ are the initial and final Ethereum states, is a SNARK that proves the following:

$$\begin{cases} S_0 \to \cdots \to S_T; \\ S_0 := (\texttt{pc}_0 := 0, \texttt{regs}, \texttt{mem}) \text{ where } \texttt{regs}, \texttt{mem} \text{ are derived from } \mathbb{E}; \\ S_T := (\texttt{pc}_T, \texttt{regs}', \texttt{mem}') \text{ where } \texttt{regs}', \texttt{mem}' \text{ are derived from } \mathbb{E}'; \end{cases}$$

1.2 Supported Instructions

SP1 supports the full RV64IM instruction set[2]: the base 64-bit integer instructions (I) and the integer multiply/divide extension (M). Additionally, SP1 provides *precompiles*—accelerated implementations of expensive operations—invoked via the RISC-V `ecall` instruction with a syscall code in register `t0`.

The precompiles include:

- **Hash functions:** SHA-256 (extend + compress), Keccak-256 permutation, Poseidon2.
- **Elliptic curve operations:** point addition, doubling, and decompression for Ed25519, Secp256k1, Secp256r1, BN-254, and BLS12-381.
- **Field arithmetic:** $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$ add/sub/mul for BN-254 and BLS12-381.
- **Big-integer arithmetic:** 256-bit multiply, 256-bit add/multiply with carry, 256×2048-bit multiply.

[2] `https://riscv.org/technical/specifications/`

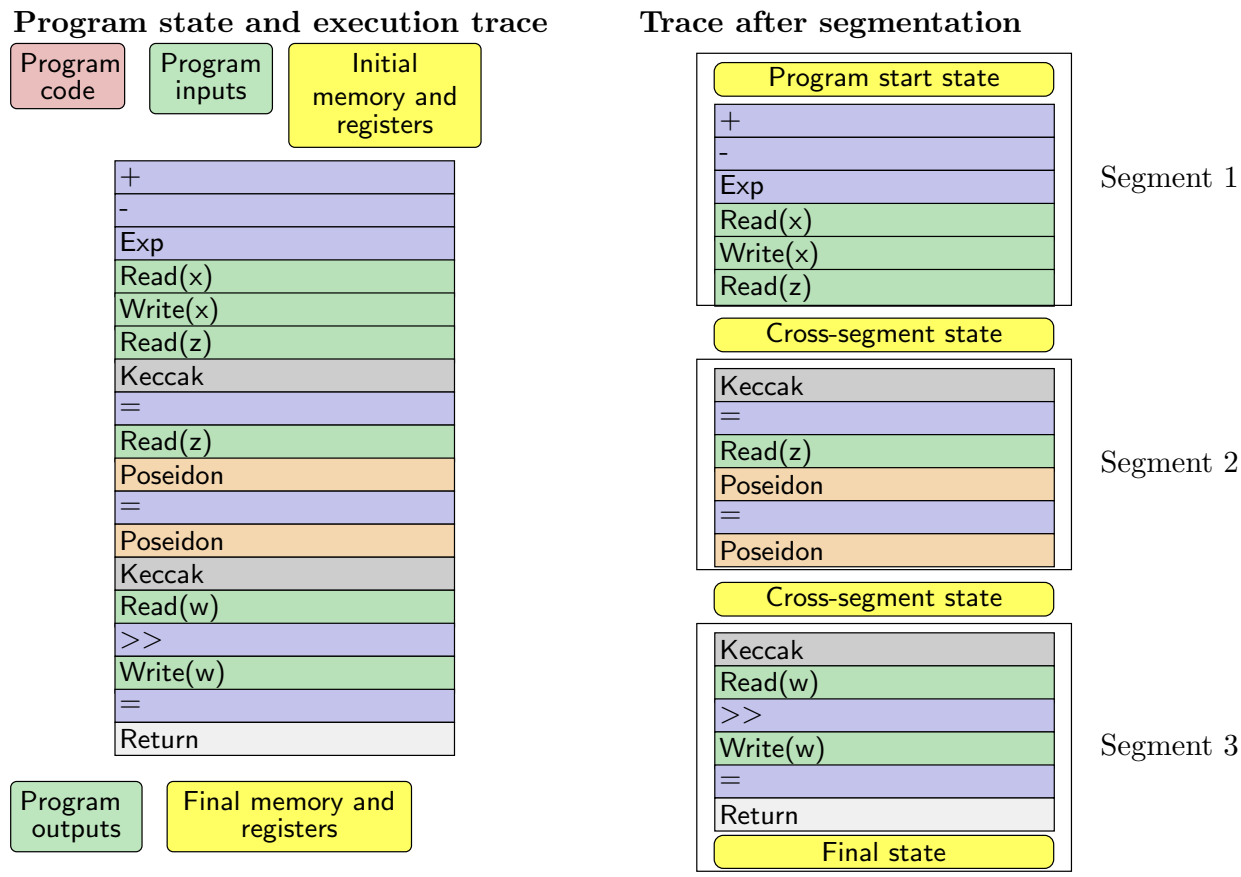

Figure 1: Segmentation of the execution trace into 3 segments

- **System:** halt, I/O (write, hint read/len), commit, deferred proof verification, memory protection.

For our purpose the exact semantics of each instruction is not important; we treat them as black boxes with predicates modeling correctness. For a given instruction op, the predicate φ_{op} has the semantics

$$\varphi_{op}(\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1, \texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2) = \text{TRUE} \iff (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1) \xrightarrow{\text{op}} (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2) \wedge \texttt{code}[\texttt{pc}_1] = op \quad (5)$$

2 Segmentation

We now explain how execution is split into shards.

2.1 Execution Shards

After execution, the execution trace $\mathcal{T}$ is divided into *shards* $\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_m$ (cf. Fig. 1). Shards are not fixed-size; instead, the executor splits the trace when the estimated trace area of the current shard exceeds a configurable threshold[3]. Each shard $\mathcal{G}_i$ corresponds to a contiguous subsequence of the execution trace, from S_{start_i} to S_{end_i}, and is proven by its own core shard proof.

[3]The default element threshold is $2^{28} + 2^{27}$, and the maximum table height per shard is 2^{22}.

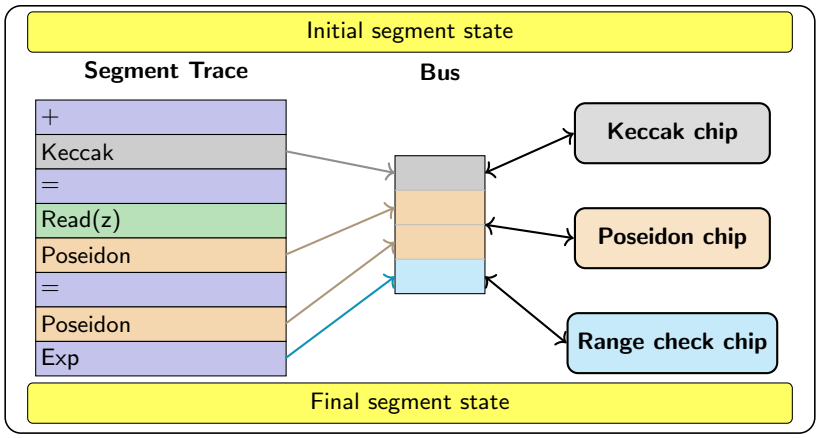

Figure 2: Segment structure: segment trace connects to the bus which then connects to chips.

Sharding reduces the proof of the entire execution to proving the validity of each shard and consistency between them. The global consistency argument (enforced during recursive aggregation) asserts:

$$S_{\mathrm{end}_i} = S_{\mathrm{start}_{i+1}} \text{ for all } i \in [m-1], \quad S_{\mathrm{start}_1} = S_0, \quad S_{\mathrm{end}_m} = S_T.$$

2.2 Shard layout

Each shard is proven by a *chip cluster*—a collection of chips (specialized constraint systems) that are included together in a single proof. The cluster determines which chips are present and thus the circuit shape. Within a shard, each chip produces a table of field elements with algebraic constraints on its columns. The full list of chip cluster types and their contents is given in Section 4.2.

At a high level, every shard includes:

- An **execution trace chip** containing the RISC-V instruction steps, register accesses, and control flow for the shard's portion of execution.
- **Precompile chips** for any accelerated operations invoked during the shard (e.g., SHA-256, Keccak, elliptic curve operations). Depending on cost, a precompile may be included in the same cluster as the execution trace or deferred to its own shard (Section 4.2).
- **Infrastructure chips** for lookups (byte, range), memory state, timestamp management, syscall dispatch, and the global bus accumulator.

Consistency between chips within a shard is enforced by a *bus*: a LogUp interaction argument (proved via GKR) that checks that all "send" and "receive" messages across chips balance within the shard.

3 Correct execution of a program

In this section, we define via predicates what it means that a program is correctly executed. We will use these predicates in Section 4 to define the relations that the zkVM proves.

3.1 Predicates for individual operations

Each operation is treated as a black box with its own predicate φ_{op} (cf. (5)). Throughout this section, subscripts 1 and 2 denote states before and after an operation. We write $S_1 := (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1)$ and $S_2 := (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$ for the states before and after the operation.

We group operations as follows:

- **ALU operations.** Integer arithmetic (ADD, SUB, MUL, DIV, REM, and their variants), bitwise operations (AND, OR, XOR), shifts (SLL, SRL, SRA), and comparisons (SLT, SLTU). Each has a predicate $\varphi_{\text{op}}(S_1, S_2)$.
- **Memory operations.** Loads (LB, LH, LW, LD and unsigned variants) and stores (SB, SH, SW, SD). Load: $\varphi_{\text{load}}(S_1, S_2)$ — reads memory into a register. Store: $\varphi_{\text{store}}(S_1, S_2)$ — writes a register value to memory.
- **Control flow.** Branches (BEQ, BNE, BLT, BGE, BLTU, BGEU), jumps (JAL, JALR), and upper-immediate (LUI, AUIPC).
- **Precompiles.** Invoked via `ecall`; each precompile P has a predicate $\varphi_{\text{P}}(S_1, S_2)$ covering all state changes (register and memory) produced by the syscall.

3.2 Correct execution of a single step

A single step executes the instruction at the current program counter. The step predicate selects the matching operation via a disjunction over all opcodes:

$$\varphi_{\text{step}}(S_1, S_2) := \bigvee_{\text{op} \in \text{operations}} \varphi_{\text{op}}(S_1, S_2)$$

where S_1 and S_2 are the states before and after the step.

3.3 Chip interactions and the bus

Within a shard, chips communicate through a *bus*: a collection of typed messages that chips *send* (produce) and *receive* (consume). For example, when the execution trace chip encounters a precompile call, it sends the call's inputs to the bus; the corresponding precompile chip receives those inputs, performs the computation, and sends the results back. Consistency is enforced by requiring that, for each interaction kind, the multiset of sent messages equals the multiset of received messages.

SP1 implements this via a **LogUp argument** (proved via GKR): each chip contributes terms $+1/(\alpha - m_i)$ for sends and $-1/(\alpha - m_j)$ for receives, where α is a random challenge and m_i, m_j are message fingerprints. The sum must equal zero, which holds if and only if sends and receives match as multisets.

Each interaction is declared with one of two *scopes*:

- **Local** interactions must balance within the shard. This is enforced by the LogUp + GKR argument inside the shard proof.
- **Global** interactions need not balance within a single shard. Instead, each shard computes its net contribution $\sigma \in E(\mathbb{F})$, a point on a degree-7 elliptic curve over the base field (the "septic curve"), and outputs it as a public value. The contributions are accumulated across all shards during recursion, and the final completeness check asserts $\sum_i \sigma_i = \mathcal{O}$ (the identity point).

The global scope is used for interactions that cross shard boundaries, most notably memory consistency and precompile delegation to separate shards.

3.4 Memory consistency

SP1 does *not* use Merkle tree commitments for memory. Instead, memory consistency is enforced entirely through the global bus.

At the beginning of execution, a dedicated `MemoryGlobalInit` chip sends each initial memory cell $(addr, value, timestamp)$ into the global bus. At the end of execution, a `MemoryGlobalFinal` chip receives the final memory state from the global bus. During execution, every load and store instruction interacts with the bus: a load receives the current value at the given address and sends an updated timestamp; a store receives the old value and sends the new value with an updated timestamp.

Because these are global-scope interactions, they accumulate across shards via the septic curve. The final completeness check ($\sigma = \mathcal{O}$) ensures that every memory cell written is eventually read with the correct value, and that the initial and final memory states are consistent with the full execution.

Within a shard, local memory interactions (via the `MemoryLocal` chip) handle the read/write operations that occur entirely within that shard, using the local LogUp argument for consistency.

4 Recursion Architecture

In the previous two sections, we described how a proof of correct RISC-V execution is produced by dividing the computation into shards, arithmetizing shard correctness via constraints and lookups, and finally gluing together the shard proofs via a global bus consistency argument. The number of shards scales roughly with the cycle count of the execution; to produce constant-size proofs it is therefore necessary to introduce some form of recursion or proof aggregation. In this section, we describe the approach taken by SP1 to address this problem.

It is useful when discussing recursion to set some terminology to avoid the confusion caused by "the recursive nature of recursion". First, by a **recursion program** or **recursion circuit**, we mean a program or arithmetic cirucit that verifies one or more SNARK proofs and also optionally checks some aggregation logic. SNARK proofs can typically be verified in the circuit model of computation, so the term "circuit" here is suitable. Given a recursion circuit $\mathcal{C}$, when we say a $\mathcal{C}$ proof, we mean a proof of the correct execution of $\mathcal{C}$. Often, the inputs to a recursion circuit are $\mathcal{C}'$ proofs for some other circuit $\mathcal{C}'$; we advise the reader to be aware that the term $\mathcal{C}$ proof refers to the *output* of proving $\mathcal{C}$. Finally, given a collection

4.1 Topology

The recursion proof pipeline consists of multiple circuit types, summarized in the table below.

Circuit	Purpose	Relation
`normalize`	1-to-1 shape normalization	$\mathcal{R}_{\text{norm}}$
`compose`	N-to-1 aggregation ($1 \leq N \leq 4$)	$\mathcal{R}_{\text{compose}}$
`shrink`	Final proof size reduction	$\mathcal{R}_{\text{shrink}}$

In more detail:

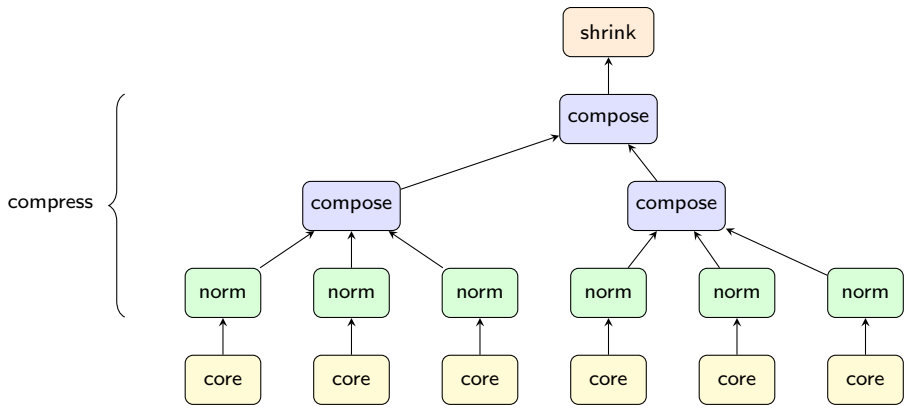

Figure 3: Recursion topology. Colors: core , normalize , compose , shrink . The tree is illustrative; the actual fan-in and depth depend on the number of shards.

- `normalize`: A 1-to-1 circuit that verifies a single core shard proof. A `normalize` proof is always of a fixed shape (which we call `compress_shape`), irrespective of the input proof's shape (Sec. 4.3).
- `compose`: An N-to-1 recursion circuit ($1 \leq N \leq 4$) that verifies multiple proofs of shape `compress shape` and performs some aggregation logic. A `compose` proof is always a single proof of shape `compress_shape` (Sec. 4.4).
- `shrink`: A 1-to-1 circuit that takes the final complete `compose` proof and re-proves it with smaller FRI parameters, reducing the proof size in preparation for the outer proof system (Sec. 4.5).

We visualize the recursion topology in Fig. 3. At the leaves, `core` shard proofs are each normalized to `compress_shape` by a `normalize` circuit. The resulting proofs are then aggregated by a tree of `compose` circuits (fan-in up to 4). We refer to the `normalize` and `compose` layers collectively as the "compress" stage. The compress stage terminates when all proofs have been aggregated into a single complete `compose` proof. Finally, this proof is passed to the `shrink` circuit, which re-proves it with reduced FRI parameters to minimize proof size before conversion to the outer proof system.

Programs vs. verifiers. Because of recursive proof aggregation, each recursion step acts as both a *prover* and a *verifier*: the program executed by the prover contains the verifier for the previous step's proof system. For example, the `compose` program contains the compress verifier, which verifies `normalize` and `compose` proofs. In the subsections below, when we say "the `compose` circuit verifies a proof," we mean that the compose *program* embeds the compress *verifier* as part of its constraint system.

Verifying key Merkle tree. Each recursion circuit type may have multiple *shapes*—distinct circuit configurations that arise from different chip clusters and trace dimensions. A verifying key vk is derived for each shape, yielding approximately 181,000 distinct keys in total. These keys are organized into a Merkle tree whose root vk_root is used to authenticate recursion proofs.

The shapes are enumerated as follows:

- **Normalize** (the vast majority of leaves): one shape per combination of chip cluster, preprocessed and main stacked BaseFold column counts, and padding column counts.

- **Compose**: one shape per arity ($N = 1, 2, 3, 4$).
- **Shrink**: a single shape.

The `compose` program enforces VK authentication: it takes Merkle membership proofs for each input proof's verifying key as part of its witness, verifies them against the Merkle root, and checks that every input proof's $\mathsf{rpv.vk_root}$ equals this root. The `normalize` program does not perform VK membership checks itself; it propagates $\mathsf{vk_root}$ as a public value, which the compose verifier then checks for consistency.

Note that, because of the 1-1 normalization, there truly is exactly one `compose` circuit for each arity. If not for the normalization, the number of arity N `compose` circuits would be C^N, where C is the number of core shard proof shapes.

4.2 `core` circuits

To optimize the proof size and prover time, we introduce core shard circuits that prove the correctness of RISC-V execution and precompile execution.

The circuits are determined by a "chip cluster," i.e. a collection of chips that are proved together in a single shard. All inter-shard communication is enabled by passing messages through the global accumulation bus, which is arithmetized as a `Global` table present in all shards.

Each chip declares interactions with two possible scopes:

- **Local** interactions must balance within the shard. This is enforced by a LogUp argument (proved via GKR) inside the shard proof.
- **Global** interactions need not balance within a single shard. Instead, each shard computes its net contribution $\sigma \in E(\mathbb{F})$, a point on a degree-7 elliptic curve over the base field (the "septic curve"), and outputs it as a public value. The contributions are accumulated across all shards during recursion, and the final completeness check asserts $\sum_i \sigma_i = \mathcal{O}$ (the identity point).

Each shard proof attests to the relation

$$\mathcal{R}^{(C)}_{\text{shard}} = \left\{ ((\mathsf{pv}, \sigma);\ \mathsf{trace}) \;\middle|\; \begin{array}{c} \text{All chip constraints in cluster } C \text{ are satisfied} \\ \text{All local-scope bus interactions balance (via LogUp)} \\ \sigma \text{ is the correct global cumulative sum for the global-scope interactions} \\ \text{Public values } \mathsf{pv} \text{ are consistent with the trace} \end{array} \right\}$$

where C is the chip cluster determining the circuit, pv are the shard's public values (boundary state, digests, etc.), and σ is the shard's global cumulative sum.

The clusters are of 4 types:

1. **Base core cluster** (`core`). Contains all RISC-V instruction chips (ALU, memory load/-store, control flow, etc.) together with infrastructure chips: `Program` (instruction lookup), `ByteLookup` and `RangeLookup` (preprocessed lookup tables), `MemoryLocal` (local memory state), `MemoryBump` and `StateBump` (timestamp management), `SyscallCore`, `SyscallInstrs` (syscall dispatch), and `Global` (global bus accumulator).
2. **Extended core clusters** (`core+`P). Some precompiles are compact enough to be retained in the core cluster, avoiding the overhead of a separate shard proof and allowing communication with their calling context via the local lookup argument. The retainable extensions are:

MemoryGlobalInit/MemoryGlobalFinal, Bls12381Fp, Bn254Fp, Sha256{Extend,Compress} (with their control chips), Uint256Ops, and Poseidon2. Extended clusters are generated for subsets of size 0, 1, or all of these extensions, plus one additional combination (core + memory global + SHA-256 + Uint256Ops).

3. **Deferred precompile clusters** (precompile-P). Precompiles with high arithmetization costs (in terms of number of columns in the arithmetization) are proved in their own shards. Each such cluster contains the precompile-specific chip(s) together with a base set of infrastructure chips: Program, ByteLookup, RangeLookup, SyscallPrecompile, MemoryLocal, and Global. They communicate with their RISC-V calling context via the global lookup argument. The deferred precompiles include: elliptic curve operations (Ed25519, Secp256k1, Secp256r1, BN-254, BLS12-381 point add/double), field arithmetic (BLS12-381 and BN-254 $\mathbb{F}_p/\mathbb{F}_{p^2}$ operations), hash functions (Keccak permutation, SHA-256, Poseidon2), and big-integer arithmetic (Uint256Mul, Uint256Ops).

4. **Memory boundary cluster** (mem-boundary). The global memory argument requires initialization and finalization: sending the initial memory state into the global bus and receiving the final memory state from the global bus. This cluster contains MemoryGlobalInit, MemoryGlobalFinal, Global, and the preprocessed chips. Note that when memory initialization/finalization is retained into an extended core cluster (type 2), this standalone cluster is not needed (this happens for very small programs so that the proof can fit into one shard).

4.3 normalize circuit

The normalize circuit proves the correct verification of a single core shard proof. Its output is a proof of a fixed shape (compress_shape), irrespective of the input shard proof's shape (which varies with the chip cluster). The circuit takes as witness a shard proof π, the verifying key vk, a completeness flag is_complete $\in \{0, 1\}$, and a VK Merkle root vk_root.

The circuit performs two categories of work:

- **Shard proof verification.** A Fiat–Shamir challenger is initialized by observing the verifying key, and the core shard proof is verified against this challenger (polynomial commitment openings and constraint satisfaction).

- **Aggregation checks.** Outside the shard verifier, the circuit enforces cross-shard and global correctness:

  - *First-shard PC validation.* If the shard is the first execution shard, the circuit asserts that pc_start matches the verifying key's start PC.
  - *Global cumulative sum accumulation.* The output global cumulative sum σ is computed by adding the verifying key's initial_global_cumulative_sum (if this is the first shard, otherwise $\mathcal{O}$) to the shard's own global_cumulative_sum.
  - *Public values propagation.* The shard's public values (boundary PCs, timestamps, memory addresses, digests, exit codes, etc.) are propagated into a RecursionPublicValues struct together with the VK digest and VK Merkle root. A Poseidon-2 digest of these values is committed as the proof's public output.
  - *Completeness conditions.* When is_complete $= 1$, the proof is expected to attest to a full program execution rather than a single shard, so final boundary conditions are checked — including that the program has halted, the first shard was included, all "previous"

boundary values are at their initial state, and the global cumulative sum equals $\mathcal{O}$ (the identity point on the septic curve).

Formally, let $\Pi^{(C)}_{\text{shard}}$ be the SNARK for $\mathcal{R}^{(C)}_{\text{shard}}$. Write pv for the shard's public values (including its global cumulative sum σ_{shard}) extracted from the proof, and rpv for the recursion public values output by the circuit.

$$\mathcal{R}_{\text{norm}} = \left\{ \mathsf{rpv};\ (\pi,\ \mathsf{vk},\ C) \;\middle|\; \begin{array}{c} \Pi^{(C)}_{\text{shard}}.\mathsf{Verify}(\mathsf{pv},\ \pi) = 1 \\ \mathsf{rpv.contains_first_shard} = 1 \implies \mathsf{pv.pc_start} = \mathsf{vk.pc_start} \\ \mathsf{rpv}.\sigma \ = \ \sigma_{\text{init}} + \mathsf{pv}.\sigma_{\text{shard}}, \quad \sigma_{\text{init}} := \begin{cases} \mathsf{vk}.\sigma_0 & \text{if first shard} \\ \mathcal{O} & \text{otherwise} \end{cases} \\ \mathsf{rpv} \text{ is correctly derived from } \mathsf{pv},\ \mathsf{vk},\ \text{and auxiliary inputs} \\ \mathsf{rpv.is_complete} = 1 \implies \varphi_{\text{complete}}(\mathsf{rpv}) \end{array} \right\}$$

where $\varphi_{\text{complete}}$ asserts the final boundary conditions described above (program halted, first shard included, initial boundary values zero, $\sigma = \mathcal{O}$).

4.4 `compose` circuit

The `compose` circuit is an N-to-1 recursion circuit ($1 \leq N \leq 4$) that consumes N proofs — each of which may be a `normalize` or `compose` proof — and produces one proof of the fixed shape `compress_shape`. Its purpose is to aggregate a batch of proofs that cover contiguous portions of the execution into a single proof.

The circuit performs two categories of work:

- **Proof verification.** Each input proof π_i is verified against its own verifying key vk_i. The circuit checks that every proof's $\mathsf{vk_root}$ matches a single witnessed value, and that each proof's recursion public values have a valid digest.
- **Continuity and aggregation checks.** The circuit enforces that the N proofs form a contiguous chain:
  - *Sequential continuity.* For each consecutive pair of proofs, the "last" boundary values of proof i (PC, timestamp, memory addresses, page indices, exit code, syscall flags, etc.) must equal the "previous" boundary values of proof $i{+}1$.
  - *Program identity.* The $\mathsf{sp1_vk_digest}$ and $\mathsf{proof_nonce}$ are asserted to be identical across all N proofs.
  - *Accumulation.* The global cumulative sums σ_i from all proofs are summed (via elliptic curve addition on the septic curve). The shard counts and $\mathsf{contains_first_shard}$ flags are accumulated.
  - *Completeness conditions.* When $\mathsf{is_complete} = 1$, the same final boundary conditions as in the `normalize` circuit are checked via $\varphi_{\text{complete}}$.

Formally, let Π_{comp} denote the SNARK for `compress_shape` proofs (used by `normalize` and `compose` alike). Write rpv_i for the recursion public values of the i-th input proof ($1 \leq i \leq N$, $1 \leq N \leq 4$), and rpv for the aggregated output.

$$\mathcal{R}_{\text{compose}} = \left\{ \begin{array}{c} \mathsf{rpv};\ (\{\mathsf{vk}_i, \pi_i\}_{i=1}^{N},\ \mathsf{vk_root}) \end{array} \middle| \begin{array}{c} \forall\, i:\ \Pi_{\text{comp}}.\mathsf{Verify}(\mathsf{rpv}_i,\ \pi_i) = 1\ \wedge\ \mathsf{rpv}_i.\mathsf{vk_root} = \mathsf{vk_root} \\ \forall\, i < N:\ \mathsf{rpv}_i.\mathsf{last_*} = \mathsf{rpv}_{i+1}.\mathsf{prev_*} \\ \forall\, i:\ \mathsf{rpv}_i.\mathsf{sp1_vk_digest} = \mathsf{rpv}_1.\mathsf{sp1_vk_digest} \\ \mathsf{rpv}.\sigma\ =\ \sum_{i=1}^{N} \mathsf{rpv}_i.\sigma \\ \mathsf{rpv} \text{ boundary values derived from } \mathsf{rpv}_1.\mathsf{prev_*} \text{ and } \mathsf{rpv}_N.\mathsf{last_*} \\ \mathsf{rpv}.\mathsf{is_complete} = 1 \implies \varphi_{\text{complete}}(\mathsf{rpv}) \end{array} \right\}$$

Here "$\mathsf{last_*} = \mathsf{prev_*}$" is shorthand for the continuity of all chained boundary values (PC, timestamp, memory init/finalize addresses, page indices, exit code, and syscall flags).

4.5 `shrink` circuit

The `shrink` circuit is a 1-to-1 recursion circuit that takes the final complete `compose` proof and re-proves it with reduced FRI parameters. Its purpose is to reduce the proof size: the compress stage uses FRI parameters sized for a log-trace-area of 27, while the shrink circuit uses parameters for a log-trace-area of 25, yielding a significantly smaller proof that is cheaper to verify in the subsequent outer proof system (e.g., Groth16). The shrink circuit has a single fixed shape.

The circuit takes as witness a compress-stage proof π, the verifying key vk of the input proof, and a VK Merkle membership proof mkl.

The circuit performs the following work:

- **VK Merkle verification.** The input proof's verifying key is checked against the VK Merkle tree, ensuring it belongs to the set of permitted recursion circuits.
- **Proof verification.** The compress-stage proof π is verified against its verifying key vk using the compress verifier (the same verifier used inside `compose`).
- **Completeness enforcement.** The circuit asserts that the input proof is marked $\mathsf{is_complete} = 1$, meaning it attests to a full program execution—the program has halted, all shards have been aggregated, the global cumulative sum equals $\mathcal{O}$, and all boundary conditions are satisfied.
- **Output construction.** The output recursion public values are derived from the input proof's public values. A Poseidon-2 digest of the output public values is committed as the shrink proof's public output.

Formally, let Π_{comp} denote the SNARK for `compress_shape` proofs (as in the `compose` circuit). Write rpv' for the recursion public values extracted from the input proof and rpv for the output.

$$\mathcal{R}_{\text{shrink}} = \left\{ \begin{array}{c} \mathsf{rpv};\ (\mathsf{vk},\ \pi,\ \mathsf{mkl}) \end{array} \middle| \begin{array}{l} \Pi_{\text{comp}}.\mathsf{Verify}(\mathsf{rpv}',\ \pi) = 1 \\ \mathsf{vk} \text{ verified against } \mathsf{mkl} \\ \mathsf{rpv}'.\mathsf{is_complete} = 1 \\ \varphi_{\text{complete}}(\mathsf{rpv}') \\ \mathsf{rpv} \text{ is correctly derived from } \mathsf{rpv}' \end{array} \right\}$$

Because shrink enforces completeness, its output serves as the definitive attestation that the entire program execution is valid. The shrink proof is then passed to the outer proof system for final verification.

5 Proof system instantiations

In the previous sections, we have described how to arithmetize the correct RISC-V execution of a program by decomposing it into a collection of tables of field elements, each of which has algebraic constraints placed on its columns. Furthermore, we allow tables to "send" and "receive" messages with multiplicity through a collection of "interaction kinds". All tables are zero padded to have the same height, call it $\mathcal{N}_{row} = 2^{n_{row}}$. For the RISC-V circuits, we use $n_{row} = 22$. The reason for this padding is that it allows for the verifier to be described as a uniform arithmetic circuit, independent of the unpadded table heights. This naturally introduces some prover and verifier overhead; naively, this can be a very large overhead if one of the tables is wide and short, in which case the padding adds a substantial number of trace cells, but the Hypercube proof system has a number of mitigations to minimize this overhead. We will describe some of these mitigations below, especially because one of them is a cryptographic protocol invented for this particular zkVM context. First, though, let us describe protocol as if it were actually run on the padded tables:

1. Treat each column of each table as a multilinear polynomial in n_{row} variables. Use a batch multilinear polynomial commitment scheme to commit to these polynomials. Let W denote the total number of columns/multilinear polynomials.

2. Use GKR with n_{row} maybe ± 1 rounds to prove the LogUp argument equating the "send" and "receive" multisets across all tables. The output of GKR is a collection of W evaluation claims, one on each column, all at the same point η.

3. Use the point η as the randomness for the zerocheck sumcheck which proves constraint satisfaction. Batch the zerocheck sumcheck with the sumchecks which "prove" the evaluation claims from LogUp+GKR (more precisely, the sumchecks reduce evaluation claims at one point to evaluation claims at another). The output of this batched sumcheck is again a collection of W evaluation claims, not at a new point η'.

4. Use the batch multilinear polynomial commitment scheme to prove these evaluation claims.

If the prover actually materialized the full $W \times \mathcal{N}_{row}$ trace, this would be an enormous quantity of data. For example $W \approx 2^{12}$ is in the mid-range of column counts among various "table clusters" used in SP1. For a 32-bit field such as KoalaBear, this translates into a total of $4 \times W \times \mathcal{N}_{row} = 2^{36}$ bytes, or 64 GB. This is twice the global VRAM of an Nvidia 5090 GPU. Clearly, keeping around the padding is not feasible. In order to mitigate these costs, the SP1 proof system uses the following techniques:

1. Instead of using a batch multilinear PCS directly, we use the jagged polynomial commitment scheme, which is a sparse PCS designed for exactly the kind of sparsity inherent in the table layout as described above. This enables the prover and verifier PCS costs to depend only on the size of the unpadded data (and notably, the verifier PCS costs can be made independent of W). For the internal dense PCS, we use batch BaseFold, which we view as a multilinear PCS for committing to a single multilinear in `log_total_unpadded_trace_area` variables. The evaluation proof for a point $\eta = \eta_n || \eta_k$ consists of

2. For the sumchecks, the contributions of the padding rows are always easy to compute and don't scale with the amount of padding. The prover makes frequent use of this fact.

Important Note: The Hypercube proof system verifier can be described as an arithmetic circuit. The circuit is determined by the total trace area of the unpadded data, and the tables

included in the segment. Notably, the circuit is *independent* of the individual tables' unpadded heights. Hence, the total number of verifier circuits can be easily limited to the $\approx 2^{18}$ range, which makes it feasible to build them all instead of using a VM for recursion.

In the following subsections, we describe each section of the protocol in a bit more detail, with an eye towards making the soundness analysis in soundcalc more transparent. Whenever a variable appears in `typewriter font`, it refers to a parameter in the "sp1.toml" file in the soundcalc repository.

5.1 LogUp + GKR Details

TODO: Ask Min about the LogUp+GKR business.

5.2 Zerocheck Details

Recall that we have a trace of height $\mathcal{N}_{row}$ and width W on which we place `num_constraints` algebraic constraints, denoted $C_0, C_1, \ldots, C_{num_constraints}$. Each C_i is a polynomial of total degree at most 3 in W variables, though almost always it depends trivially on most of the variables. It is useful also to write $W = w_1 + \ldots + w_k$, where k is the number of tables in the cluster, and to group the constraints by table, so that $C_{i,j}$ refers to the j-th constraint on the i-th table. Let us write c_i for the number of constraints in the i-th table, and $p_{i,\ell}$ for the ℓ-th column of the i-th table. Finally, let us write h_i for the height of the i-th table (not including padding). The claim for our zerocheck is that, $\forall x \in \{0,1\}^{n_{row}}, \quad i \in [k], \quad j \in [c_i]$:

$$C_{i,j}(p_{i,1}[x], \ldots, p_{i,w_i}[x]) = \begin{cases} 0, & x < h_i, \\ C(0, \ldots, 0), & x \geq h_i \end{cases} \tag{6}$$

Note that the constraint evaluation on zero-padding may not result in 0, so at the moment our protocol does not match exactly with the usual zerocheck. However, note the following. If $\mathrm{geq}_{h_i}(x)$ is the function which, for $x \in \{0,1\}^{n_{row}}$, is zero if $x < h_i$ and 1 if $x \geq h_i$, then its multilinear extension $\widehat{\mathrm{geq}}_{h_i}(z)$ is relatively straightforward to evaluate at arbitrary $z \in \mathbb{F}^{n_{row}}$, using the following recursion relation. Write $z = (z_0, z_{rest}) \in \mathbb{F} \times \mathbb{F}^{n_{row}-1}$, and write $h_i = h_{i,0} * 2^{n_{row}-1} + h_{i,rest}$. Then, we have:

$$\widehat{\mathrm{geq}}_{h_i}(z) = z_0 \cdot (1 - h_{i,0}) + \widehat{\mathrm{eq}}(z_0, h_{i,0}) \cdot \widehat{\mathrm{geq}}_{h_{i,rest}}(z_{rest})$$

To prove this, first, we sample a random $\zeta \in \mathbb{F}^{n_{row}}$, $\lambda \in \mathbb{F}, \alpha \in \mathbb{F}$, and turn this collection of claims into the single sumcheck claim:

$$\sum_{i=0}^{k-1} \sum_{j=0}^{c_i-1} \lambda^i \alpha^j \widehat{\mathrm{geq}}_{h_i}(\zeta) = \sum_{x \in \{0,1\}^{n_{row}}} \widehat{\mathrm{eq}}_\zeta(x) \sum_{i=0}^{k-1} \lambda^i \sum_{j=0}^{c_i-1} \alpha^j C_{i,j}(p_{i,1}[x], \ldots, p_{i,w_i}[x]) \tag{7}$$